

PERSONAL TECH STACK

What I've Built

A plain-language walkthrough of the hardware, AI, and automation I run day to day — for explaining it to people, not just for me.

Carlos Lucero • Princeton, TX
As of August 2026

I run two things off the same desktop PC: a public AI news business (Prompt AI News, promptainews.com) that publishes semi-autonomously with me as the final approval, and a private local AI lab — two chatbots I built for myself that never touch the cloud. This document is the plain-English version I hand someone who asks “wait, what exactly do you have running?” No credentials or tokens are included here on purpose.

A Hardware

WHAT IT IS

The physical machine everything else runs on.

HOW I USE IT

One Windows 11 desktop: Intel Core i5-14400F CPU, NVIDIA RTX 5060 with 8GB of VRAM. The GPU is what makes local AI possible at all — it's what runs the chatbots without sending anything to the cloud. My iPhone reaches the desktop remotely over Tailscale (a private VPN mesh, IP 100.85.231.11), and Windows Task Scheduler acts as the “cron” for everything that needs to run on a timer.

GOOD TO KNOW

8GB of VRAM is a real ceiling, not a preference — it caps me to roughly 8-billion-parameter models running in a compressed (“Q4”) format with about a 16,000-token context window. That's the budget every local AI decision below has to fit inside.

B Local AI — Private Chatbots

WHAT IT IS

AI that runs entirely on my own hardware, with nothing sent to a third party.

HOW I USE IT

Ollama is the engine that loads and runs models locally (currently: hermes3, llama3.1:8b, qwen2.5:7b, dolphin3, and qwen3:14b — that last one is installed but too large to actually run on this GPU). Two chatbots sit on top of it:

- **Katie** — a custom persona built on the dolphin3:8b model via a Modelfile (a config that defines a model's personality and response style). Purely local, private chat.
- **Hermes Agent** (from Nous Research) — an agentic command-line assistant that can browse the terminal, files, and its own to-do list, plus run scheduled jobs. It runs hermes3 and connects to both WhatsApp and Telegram as chat interfaces. Two of its cron jobs run daily: a 7am NYT news digest and a 5:30am morning briefing (AI news bucketed by topic, plus a rotating 60-topic “learning explainer” written fresh each morning).

C Cloud AI — Claude / Anthropic

WHAT IT IS

AI that runs on Anthropic's cloud compute — used when a task needs more capability than the local 8GB GPU can offer, or needs to reach the internet.

HOW I USE IT

Claude Code is where I do the actual engineering and writing work for Prompt AI News (including producing this document). Separately, I have two scheduled **cloud routines** — Claude agents that wake up on a timer with no laptop or app open: one publishes Prompt AI News drafts every morning at 6:30 AM CDT, and another reminds me to place my weekly Walmart grocery order every Sunday at 6 PM CDT. The publisher routine is connected to Gmail, Google Calendar, Google Drive, and Notion so it can read my morning digest and act on it directly.

D Prompt AI News — Production Stack

WHAT IT IS

The actual website and the automation that feeds it, running semi-autonomously with me as the final gatekeeper.

HOW I USE IT

The site (promptainews.com) is built on **Next.js** + **Tailwind CSS** + **TypeScript**, hosted on **Vercel** (every push to the code automatically deploys). Content lives in **Sanity**, a headless CMS — I review and publish every story through Sanity Studio before it goes live. Behind the scenes, three Python scripts scan 12+ news sources every 3 hours on Windows Task Scheduler (general AI, China AI, and EV news), and a daily analytics script emails me traffic numbers. For search visibility, the site pings Bing directly after every publish (IndexNow) and maintains a handful of “evergreen” reference pages meant to get cited by AI answer engines like ChatGPT and Perplexity, not just ranked by Google.

E Observability Stack

WHAT IT IS

The dashboards and alerts that tell me whether the local chatbot stack is actually healthy — built as a hands-on learning project alongside the chatbots themselves.

HOW I USE IT

Docker containers run the supporting services and restart themselves automatically if the PC reboots. Every request is instrumented with **OpenTelemetry**, which feeds two places: **Jaeger** (for tracing exactly what happened on a single request) and **Prometheus** (for aggregate metrics over time), both visualized in **Grafana** dashboards. A small Python monitor watches for health problems and pushes alerts straight to a Telegram bot on my phone.

F Messaging & Notification Glue

WHAT IT IS

The plumbing that lets automation actually reach me, instead of quietly running in the background.

HOW I USE IT

Gmail (SMTP) sends the news digests and analytics emails. **TextBelt** sends SMS text alerts for things like “drafts are ready to review” (I’m considering replacing this with Telegram, which is free and unlimited instead of quota-metered). **Telegram** and **WhatsApp** double as both chatbot interfaces (for Katie and Hermes) and alert channels (for the observability stack). **Umami**, self-hosted on Railway, handles website analytics without selling visitor data to anyone.

+ How It All Fits Together

It’s really two parallel systems sharing one PC. The **public side** is Prompt AI News: news comes in on a schedule, gets rewritten by Claude into journalist-quality drafts, lands in Sanity as unpublished drafts, and waits for me to personally read and approve it before anything goes live. The **private side** is Katie and Hermes Agent — local, GPU-bound chatbots with no cloud dependency, doing scheduled jobs and private conversation instead of

publishing anything. Both sides lean on the same instincts: something runs on a schedule (Task Scheduler or cron), something watches whether it's actually working (logs, Grafana, or a text alert), and a human — me — stays in the loop at the one step that matters most: what actually goes public.

This document intentionally leaves out API keys, passwords, and tokens. The full technical reference with credentials lives in a private System Bible document, not here.